Manual técnico: Aprendizaje Federado

Alejandro Martínez Orozco y Jon Henry Vásquez Mesa

Análisis y Diseño de Algoritmos · Grupo 2

Universidad de Caldas

6 de noviembre de 2026

Índice

1. Propósito y alcance **2**

2. Contexto e investigación previa **2**
2.1. Por qué el problema existe . . . 2
2.2. Qué se sabe de cada parte . . . 2
2.3. Qué decisiones tomamos a partir de eso . . . 3

3. Problema formalizado (resumen) **3**

4. Decisiones de modelado y su justificación **3**
4.1. Utilidad de reparto: $u_i(x) = a_i \ln(1 + x)$ . . . 3
4.2. Utilidad conjunta: definición explícita de f . . . 4
4.3. Otras decisiones . . . 4

5. Algoritmos **4**
5.1. Reparto voraz con montículo (Parte 1) . . . 4
5.2. Selección voraz con cupo y costos (Parte 2) . . . 5
5.2.1. Corrección 1: añadir el mejor nodo individual (Khuller, Moss y Naor, 1999) 5
5.2.2. Corrección 2: enumerar los tripletes iniciales (Sviridenko, 2004) . . . 6
5.2.3. Qué hace entonces la solución base . . . 6

6. Análisis de complejidad **6**

7. Estructura del código **7**

8. Formatos de entrada y salida **7**
8.1. Entrada (JSON) . . . 7
8.2. Salida . . . 7

9. Pasos de ejecución **8**

10. Ejemplo trabajado con el caso de la docente **8**

11. Validación realizada **9**

12. Limitaciones y trabajo futuro **10**

13. Referencias **11**

1. Propósito y alcance

La solución base resuelve las dos partes del problema para el caso de la docente (`data/N15.json`):

- **Parte 1, reparto del presupuesto de ruido:** dados los nodos ya elegidos y un presupuesto B, repartirlo para maximizar $\sum_i u_i(x_i)$.
- **Parte 2, selección de participantes:** elegir en cada ronda el subconjunto de nodos que maximiza una utilidad conjunta submodular, con un cupo k y un presupuesto de costos de participación C.

Las dos partes se encadenan a lo largo de T rondas. La selección decide quién participa, el reparto decide cuánto ruido gasta cada uno y el presupuesto total de cada nodo, P_i, se va gastando de una ronda a otra.

Quedan para etapas posteriores del cronograma los fallos de nodos, las externalidades de red en el reparto, el algoritmo exacto con su $T(n)$, la cota de aproximación con costos y la validación experimental con instancias crecientes.

2. Contexto e investigación previa

2.1. Por qué el problema existe

En aprendizaje federado (McMahan et al., 2017) varios nodos entrenan un modelo común sin compartir sus datos crudos, y solo envían actualizaciones. Esas actualizaciones todavía pueden filtrar información, así que se protegen con privacidad diferencial, que añade ruido calibrado (Dwork y Roth, 2014). Más ruido protege más, pero reduce la utilidad de la actualización, y cada nodo tolera solo una cantidad total de ruido (su presupuesto de privacidad). En cada ronda hay entonces dos decisiones: quién participa y cuánto de su presupuesto gasta.

2.2. Qué se sabe de cada parte

Pregunta	Resultado conocido	Fuente
Repartir B unidades discretas entre N funciones separables, cóncavas y crecientes	Es un problema de asignación de recursos con solución exacta en tiempo polinomial: el voraz por ganancia marginal es óptimo	Ibaraki y Katoh (1988)
Elegir k elementos para maximizar una función monótona y submodular	NP-difícil; el voraz garantiza $\geq (1 - 1/e) \approx 63{,}2\,\%$ del óptimo	Nemhauser, Wolsey y Fisher (1978)
¿Se puede mejorar esa garantía?	No, salvo P = NP, ni siquiera para Cobertura Máxima	Feige (1998)
Con costos y presupuesto (*budgeted maximum coverage*)	El voraz simple pierde la garantía; hay algoritmos que la recuperan	Khuller, Moss y Naor (1999); Sviridenko (2004)
Dureza de la selección	Reducción desde Cobertura Máxima, a su vez desde Cobertura de Conjuntos	Karp (1972); `Formalización.pdf`, Teorema de NP-dureza

Cuadro 1: Resultados conocidos que sustentan el diseño de la solución.

2.3. Qué decisiones tomamos a partir de eso

- La Parte 1 se resuelve de forma exacta con el voraz de montículo. Existe un algoritmo óptimo y eficiente, así que no tiene sentido recurrir a fuerza bruta ni a metaheurísticas.
- La Parte 2 usa un voraz: la selección es NP-difícil y la función es submodular, que es el caso en el que Nemhauser et al. dan garantía. Mientras N lo permite, comparamos además con el óptimo por fuerza bruta.
- El caso trae costos distintos además del cupo, y con ellos la cota $(1 - 1/e)$ no aplica tal cual. Por eso implementamos dos variantes del voraz y damos la comparación con el óptimo solo como referencia (sección 12).

3. Problema formalizado (resumen)

La formalización completa (entradas, salidas, tamaño n, objetivo, restricciones, casos límite) está en `doc/Formalización.pdf`. Para el caso G2, por ronda t:

- **Entrada:** N nodos, cupo k, costos c_i, presupuesto de participación C, ruido B, presupuestos totales P_i, grafo de influencia w_{ij}, funciones u_i y f.
- **Decisión:** $S_t \subseteq V$ y $x^t \in \mathbb{Z}_{\geq 0}^{S_t}$.
- **Restricciones:** $|S_t| \leq k$; $\sum_{i \in S_t} c_i \leq C$; $\sum_i x_i^t \leq B$; $\sum_t x_i^t \leq P_i$.
- **Objetivo:** maximizar $\sum_t \left[f(S_t) + \sum_{i \in S_t} u_i(x_i^t) \right]$.
- **Tamaño de la entrada:** $n = N$ (el grafo del caso es completo, así que $|E| = N(N-1)/2 = 105$; k, B, C, T se reportan aparte).

Para el caso: $N = 15$, $T = 3$, $k = 5$, $B = 30$, $C = 24$, $P_i = 18$ para todos, costos entre 2 y 7.

4. Decisiones de modelado y su justificación

Se eligieron las siguientes fórmulas de utilidad:

4.1. Utilidad de reparto: $u_i(x) = a_i \ln(1 + x)$

- a_i es `coeficiente_utilidad`.
- Es creciente y cóncava, con $u_i(0) = 0$, como exige el modelo.
- Ganancia marginal de la unidad $x + 1$:

$$\Delta_i(x + 1) = a_i \ln \frac{x + 2}{x + 1},$$

  estrictamente decreciente en x. Es un caso de rendimientos decrecientes: las primeras unidades de presupuesto aportan más que las siguientes.
- Se usa `log1p(1/(x+1))` para calcularlo con estabilidad numérica.

4.2. Utilidad conjunta: definición explícita de f

Dados un conjunto de objetivos $J = \{1, \ldots, m\}$, pesos $\beta_j \geq 0$ y probabilidades $w_{ij} \in [0, 1]$:

$$f(S) = \sum_{j \in J} \beta_j \left(1 - \prod_{i \in S} (1 - w_{ij})\right).$$

Es una cobertura probabilística ponderada: cada nodo seleccionado i «cubre» al objetivo j con probabilidad w_{ij}, y cubrir a j vale β_j. En el caso G2, $J = V$ (así $m = N$), $\beta_j = a_j$, w_{ij} es la `peso_influencia` del grafo y $w_{jj} = 1$ (un nodo seleccionado se cubre a sí mismo). Con $w_{ij} \in \{0, 1\}$ y $\beta_j = 1$ se obtiene $\left|\bigcup F_i\right|$, la Cobertura Máxima.

Por qué es monótona y submodular. Sea $\mu_S(j) = \prod_{i \in S}(1 - w_{ij}) \in [0, 1]$, la probabilidad de que j siga sin cubrir. Agregar un nodo i a S aporta

$$f(S \cup \{i\}) - f(S) = \sum_j \beta_j \, \mu_S(j) \, w_{ij} \geq 0,$$

lo que prueba monotonía. Si $A \subseteq B$, entonces $\mu_B(j) \leq \mu_A(j)$ para todo j, porque se multiplican más factores en $[0, 1]$. Por tanto, la ganancia de i sobre A es mayor o igual que sobre B, que es la submodularidad. Guardando el vector $\mu(\cdot)$, las ganancias se calculan de forma incremental.

Esta f encaja con la reducción de NP-dureza, ya que la cobertura $\left|\bigcup F_i\right|$ es el caso $w_{ij} \in \{0, 1\}$. El grafo del caso influye en la selección a través de f.

4.3. Otras decisiones

- La influencia es simétrica ($w_{ij} = w_{ji}$), como en el caso, que trae aristas no dirigidas.
- No modelamos fallos de nodos, porque el caso no trae probabilidades q_i. Un nodo está disponible mientras le quede presupuesto.
- El presupuesto es entero y discreto, y el reparto asigna las unidades de una en una.
- Cupo y costos rigen a la vez: el caso da $k = 5$ y $C = 24$, y se exigen ambos.
- El presupuesto P_i se gasta entre rondas y no se transfiere entre nodos.
- Los empates se resuelven por menor índice de nodo, para que la ejecución sea determinista.

5. Algoritmos

5.1. Reparto voraz con montículo (Parte 1)

```
reparto(S, B, restante):
    x[i] ← 0  para i en S
    H ← montículo máximo con (Δ_i(1),  i) para i en S con restante[i] > 0
    repetir B veces:
        si H vacío: parar
        (g, i) ← extraer máximo de H
        x[i] ← x[i] + 1
        si x[i] < restante[i]: insertar (Δ_i(x[i] + 1),  i) en H
    devolver x
```

Corrección (argumento de intercambio). Sea x^* una solución óptima con $\sum x_i^* = B$ que respeta los topes, y supongamos que no es la del voraz. Existen i, j con $x_i^* > g_i$ y $x_j^* < g_j$, donde g es la salida del voraz, tales que la última unidad de i en x^* tiene ganancia $\Delta_i(x_i^*)$ y

la siguiente de j, $\Delta_j(x_j^* + 1)$. El voraz tomó antes la unidad $x_j^* + 1$ de j que la unidad x_i^* de i, por lo que $\Delta_j(x_j^* + 1) \geq \Delta_i(x_i^*)$. Mover una unidad de i a j (los topes se respetan, pues $x_j^* + 1 \leq g_j \leq \mathit{restante}[j]$) no disminuye la utilidad total. Repitiendo, se llega a g sin perder optimalidad. La concavidad (ganancias no crecientes) garantiza que el orden de ganancias de cada nodo es el orden en que se asignan las unidades. Los topes P_i (cotas superiores enteras) no rompen el argumento.

5.2. Selección voraz con cupo y costos (Parte 2)

```
seleccion(elegibles, modo):
    S ← ∅ ; costo ← 0 ; μ[j] ← 1 para todo j
    mientras |S| < k:
        cand ← { i ∈ elegibles \ S : costo + c_i ≤ C }
        si cand vacío: parar
        para i en cand: g_i ← Σ_j β_j · μ[j] · w_ij    # ganancia marginal
        elegir i^* = arg max g_i                (modo "ganancia")
               o  arg max g_i / c_i             (modo "razón")
        S ← S ∪ {i^*} ; costo ← costo + c_i^*
        μ[j] ← μ[j] · (1 − w_i^*,j) para todo j
    devolver S
```

En cada ronda se ejecutan los dos modos y se conserva el de mayor $f(S)$.
Qué garantiza y qué no.

- Sin costos ($c_i = 1$, $C = k$), el voraz por ganancia cumple $f(S) \geq (1 - 1/e)\, f(S_{\text{opt}})$ (Nemhauser et al., 1978). La idea es que, en cada paso, la ganancia marginal del voraz es al menos $1/k$ de lo que falta para llegar al óptimo, lo que da

$$f(S_k) \geq \big(1 - (1 - 1/k)^k\big) f(S_{\text{opt}}) \geq (1 - 1/e) f(S_{\text{opt}}).$$

- Con costos distintos **no hay garantía**. Ejemplo: $C = 10$, un nodo A de ganancia 1 y costo 1, y un nodo B de ganancia 1,1 y costo 10. El voraz por ganancia toma B y agota el presupuesto, mientras que el óptimo toma A y otros nodos baratos por mucho más; con ganancias de ese estilo la razón puede empeorar tanto como se quiera. El voraz por razón evita este caso, pero tiene su propio contraejemplo: un nodo muy barato y de poca ganancia le gana a uno de gran valor que sí cabía.

- Para *budgeted maximum coverage* hay garantías que salen de dos correcciones al voraz por razón. La solución base no implementa ninguna, y quedan para la etapa del algoritmo aproximado. Se explican a continuación.

5.2.1. Corrección 1: añadir el mejor nodo individual (Khuller, Moss y Naor, 1999)

Qué es. Además del voraz por razón ganancia/costo, se calcula el mejor nodo individual: el nodo i con $c_i \leq C$ que maximiza $f(\{i\})$. La respuesta es el mejor entre la solución del voraz y ese nodo solo. Cuesta N evaluaciones de f más, $O(N^2)$ con esta f.

Por qué hace falta. El voraz por razón puede llenar el presupuesto con nodos baratos y dejar fuera un nodo caro que valía muchísimo. Ejemplo con $C = 10$:

El voraz por razón toma A primero, que tiene la razón mayor. Después B ya no cabe ($1 + 10 > 10$), así que obtiene 2 frente a un óptimo de 10. Si B costara C y valiera C, la razón entre lo obtenido y el óptimo sería $2/C$, tan mala como se quiera. El mejor nodo individual arregla este caso, porque B solo vale 10. Con esta corrección se prueba una garantía de $(1-1/e)/2 \approx 0{,}316$ del óptimo.

Nodo	Ganancia	Costo	Razón
A	2	1	2,0
B	10	10	1,0

Cuadro 2: Ejemplo en el que el voraz por razón falla.

5.2.2. Corrección 2: enumerar los tripletes iniciales (Sviridenko, 2004)

Qué es. Se prueba cada subconjunto de hasta 3 nodos que quepa en C como semilla. Para cada semilla se completa con el voraz por razón sobre los nodos restantes, y se conserva la mejor solución de todas. Son $O(N^3)$ semillas.

Por qué ayuda. El óptimo contiene algunos nodos de mucho valor. Entre todas las semillas de tamaño 3 hay una que coincide con los 3 nodos de mayor valor del óptimo, y al voraz solo le queda completar el resto, donde sus errores pesan menos. La garantía sube a $(1 - 1/e) \approx 0{,}632$, igual que sin costos.

Costo. Cada una de las $O(N^3)$ semillas lanza un voraz de $O(k \cdot N^2)$ con esta f, o sea $O(k \cdot N^5)$ en total. Es polinomial pero caro, y por eso no está en la solución base.

5.2.3. Qué hace entonces la solución base

Ejecuta el voraz por ganancia y el voraz por razón, y se queda con el mejor. Eso no equivale a ninguna de las dos correcciones, porque no mira el mejor nodo individual ni enumera semillas. Por eso no podemos afirmar ninguna de las dos garantías. En el caso entregado el resultado coincide con el óptimo (razón 1,0), pero es un resultado de esa instancia, no una garantía. Añadir el mejor nodo individual es barato (N evaluaciones por ronda) y es el siguiente paso.

6. Análisis de complejidad

Notación: N nodos, k cupo, B presupuesto de ruido por ronda, T rondas.

Reparto (por ronda). Construir el montículo: $O(|S|)$ con $|S| \leq k$. Cada una de las B extracciones y reinserciones cuesta $O(\log |S|)$. Total: $O(k + B \log k)$. Es polinomial en B y N, y pseudo-polinomial en la longitud de la codificación binaria de B (el informe lo aclara). En la formulación del curso, con $S = N$: $T(N, B) = O(B \log N)$.

Selección (por ronda).

- Ganancia marginal de un candidato con esta f: recorre los N nodos $j \to O(N)$.
- Un paso evalúa hasta N candidatos $\to O(N^2)$; actualizar $\mu(\cdot)$ cuesta $O(N)$.
- k pasos $\to O(k \cdot N^2)$; dos modos $\to$ factor constante 2.
- Como f está definida explícitamente, la ganancia marginal cuesta $O(m)$, y aquí $m = N$. En general el costo es $O(k \cdot N \cdot m)$. No usamos el $O(k \cdot N)$ que daría un oráculo $O(1)$, porque ya no hay oráculo, y la $T(n)$ experimental se contrastará con esta expresión.

Total. T rondas: $O\big(T \cdot (k \cdot N^2 + k + B \log k)\big)$. Con los parámetros del caso fijos es $O(N^2)$, y el caso real se resuelve en cerca de un milisegundo.

Fuerza bruta de verificación. Enumera $\sum_{r \leq k} \binom{N}{r}$ subconjuntos y evalúa f en $O(r \cdot N)$ cada uno. Para $N = 15$, $k = 5$ son 4 944 subconjuntos por ronda. Se omite si hay más de 300 000, y `--sin-exacto` la omite siempre.

7. Estructura del código

```
src/red.py            solución base (un solo módulo, sin dependencias)
tests/test_red.py     pruebas automáticas
data/N15.json         caso de la docente
resultados/           resultado.json y traza.txt por caso
doc/                  informe (.pdf y .tex) y este manual
```

Funciones de `src/red.py`:

Elemento	Responsabilidad
`Instancia`	Lee el JSON; construye la matriz de influencia (simétrica, $w_{ii} = 1$); expone `ganancia_unidad`, `utilidad_nodo` y `f`
`seleccion_voraz(inst, elegibles, modo)`	Voraz con cupo y costos; devuelve S y traza de cada paso con todos los candidatos
`seleccion_exacta(inst, elegibles)`	Fuerza bruta de verificación
`reparto_voraz(inst, S, B, restante)`	Voraz de montículo respetando topes; devuelve x y traza por unidad
`resolver(inst)`	Orquesta las T rondas, actualiza presupuestos restantes y calcula la utilidad total
`verificar(...)`	Comprueba restricciones y, si hay exacto, la razón al óptimo
`texto(...)`, `main()`	Traza legible y línea de comandos

Cuadro 3: Responsabilidades de las funciones principales.

8. Formatos de entrada y salida

8.1. Entrada (JSON)

```
{
  "parametros": {
    "numero_nodos": 15, "numero_rondas": 3,
    "cupo_participantes_por_ronda_k": 5,
    "presupuesto_ruido_por_ronda_B": 30,
    "presupuesto_costos_participacion_por_ronda": 24
  },
  "grafo": [ {"u": "N01", "v": "N02", "peso_influencia": 0.25}, ... ],
  "nodos": [ {"id": "N01", "coeficiente_utilidad": 8.5,
              "presupuesto_total_nodo": 18, "costo_participacion": 3}, ... ]
}
```

Todos los campos son obligatorios. El programa comprueba que `numero_nodos` coincida con la lista de nodos. Cualquier otro error de formato termina con la excepción de Python correspondiente, porque no hay validación de esquema más fina.

8.2. Salida

`resultados/<caso>_resultado.json`:
`resultados/<caso>_traza.txt` contiene lo mismo en texto legible.

Campo	Contenido
`tiempo_segundos`, `utilidad_total`	Tiempo del algoritmo (sin la fuerza bruta) y $\sum_t \left[f(S_t) + \sum_i u_i \right]$
`rondas[].seleccion`, `f_S`, `modo_seleccion`, `f_por_modo`	Nodos elegidos, su utilidad, el modo ganador y el valor de ambos modos
`rondas[].traza_seleccion`	Por paso: nodo elegido, ganancia marginal, costo y ganancia de **todos** los candidatos
`rondas[].reparto`, `utilidad_reparto`	Unidades por nodo y utilidad $\sum_i u_i(x_i)$
`rondas[].traza_reparto`	Por unidad: nodo que la recibe y su ganancia marginal
`rondas[].presupuesto_restante_por_nodo`	P_i menos lo gastado hasta esa ronda
`rondas[].exacto`	Óptimo por fuerza bruta, razón voraz/óptimo y la cota $1 - 1/e$
`rondas[].verificacion`	Cada restricción con OK/FALLA

Cuadro 4: Campos del archivo de resultados.

9. Pasos de ejecución

Requisitos: Python 3.8 o superior (probado en 3.14). No hay que instalar nada (`requirements.txt` no tiene dependencias).

```
git clone <repositorio> && cd federated-learning-optimization
python3 src/red.py data/N15.json      # un caso
python3 src/red.py caso1.json caso2.json -o salida   # varios casos, otra carpeta
python3 src/red.py caso.json --sin-exacto            # sin fuerza bruta (instancias
    grandes)
python3 tests/test_red.py                            # pruebas
```

Cada ejecución imprime la traza y escribe los dos archivos por caso en la carpeta de salida (por defecto `resultados/`). El resultado no depende del directorio desde el que se lance, y dos ejecuciones producen trazas idénticas.

10. Ejemplo trabajado con el caso de la docente

Datos de la ronda 1 (los valores salen de `resultados/`).

Paso	Elegido	Ganancia marginal	Costo acumulado	Observación
1	N11	103,490	4	Mayor influencia sobre los demás (pesos de hasta 0,8), pese a tener el menor coeficiente propio (7,6)
2	N14	24,278	7	
3	N09	9,933	10	
4	N15	5,531	16	
5	N13	2,348	23	Se alcanza $k = 5$; costo $23 \leq 24$

Cuadro 5: Traza de selección de la ronda 1.

Selección. Las ganancias decrecen paso a paso, como corresponde a una función submodular. Comprobación manual del paso 1:

$$f(\{\text{N11}\}) = a_{\text{N11}} + \sum_{j \neq \text{N11}} a_j \, w_{\text{N11},j} = 103{,}49.$$

El voraz por razón ganancia/costo da $f = 145{,}106$ y el voraz por ganancia da 145,580, así que se conserva este último. La fuerza bruta (4 944 subconjuntos) encuentra el mismo conjunto óptimo, con $f^* = 145{,}580$ y razón 1,0.

Reparto de $B = 30$. La primera unidad va a N14 con ganancia $10{,}1 \ln 2 = 7{,}0008$ (la mayor entre los cinco), la segunda a N09 con $9{,}9 \ln 2 = 6{,}8622$, y así sucesivamente. Resultado: $N11 = 5$, $N14 = 7$, $N09 = 7$, $N15 = 6$, $N13 = 5$ (suma 30), con utilidad 88,564.

Efecto del presupuesto total. Tras la ronda 1, N09 y N14 conservan 11 unidades de 18, y tras la ronda 2 conservan 4. En la ronda 3 solo pueden recibir 4, así que el reparto cambia a $N11 = 8$, $N14 = 4$, $N09 = 4$, $N15 = 6$, $N13 = 8$ (utilidad 85,611, menor que en las rondas anteriores porque los topes obligan a mover unidades a nodos de menor ganancia). Al terminar, los cinco nodos elegidos quedan con 0 de presupuesto.

Utilidad total del caso: 699,477 en 0,001 s.

11. Validación realizada

1) **Verificaciones en cada ejecución** (salen en la traza): $|S| \leq k$, $\sum c_i \leq C$, $\sum x_i \leq B$, $x_i \leq$ presupuesto restante y, como referencia, $f(S_{\text{voraz}}) \geq (1 - 1/e)\, f(S_{\text{opt}})$. Todas dan OK en las tres rondas.

2) **Pruebas automáticas** (`tests/test_red.py`):

   - el reparto voraz coincide con la enumeración exhaustiva de todos los repartos de 8 unidades entre 3 nodos;
   - f es monótona y submodular en un ejemplo concreto;
   - se cumplen las restricciones en las 3 rondas y ningún nodo supera su presupuesto total de 18.

3) **Comparación con el óptimo exacto:** razón voraz/óptimo = 1,0 en las tres rondas. Con costos esto es una observación sobre esta instancia, no una garantía.

4) **Reproducibilidad:** ejecutamos desde otro directorio y con otra carpeta de salida, y la traza fue la misma.

12. Limitaciones y trabajo futuro

Limitación	Consecuencia	Etapa donde se aborda
u_i y f son elección nuestra (sección 4)	Los valores numéricos dependen de ella, la corrección del algoritmo no	Confirmar con la docente
Con costos distintos no hay garantía $(1 - 1/e)$	La razón al óptimo es solo referencial; se necesita el voraz con mejor elemento individual o enumeración de tripletes	Algoritmo aproximado (semanas 7-8)
Sin fallos de nodos	No se modela la variante de rondas con fallos	Variantes
El reparto ignora las externalidades de red	Con utilidades dependientes de vecinos, el voraz del reparto puede dejar de ser óptimo	Variantes
Selección $O(k \cdot N \cdot m) = O(k \cdot N^2)$ con f explícita	La $T(n)$ experimental se compara con esta expresión	Experimentación (semanas 9-10)
Fuerza bruta solo para N pequeño	No sirve como referencia con instancias grandes	Algoritmo exacto (semanas 5-6)
Selección miope entre rondas	No se planifica el uso del presupuesto P_i a lo largo de las T rondas: el voraz elige igual todas las rondas	Variantes
Validación de entrada mínima	Un JSON mal formado produce una excepción de Python	Sin etapa asignada
Una sola medición de tiempo	Es válida como evidencia de ejecución, pero es ruidosa	Experimentación

Cuadro 6: Limitaciones de la solución base y su tratamiento posterior.

13. Referencias

- B. McMahan, E. Moore, D. Ramage, S. Hampson y B. Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data.* AISTATS, 2017.
- C. Dwork y A. Roth. *The Algorithmic Foundations of Differential Privacy.* Foundations and Trends in Theoretical Computer Science 9(3–4), 2014.
- T. Ibaraki y N. Katoh. *Resource Allocation Problems: Algorithmic Approaches.* MIT Press, 1988.
- G. L. Nemhauser, L. A. Wolsey y M. L. Fisher. *An analysis of approximations for maximizing submodular set functions-I.* Mathematical Programming 14, 1978.
- U. Feige. *A threshold of* $\ln n$ *for approximating set cover.* Journal of the ACM 45(4), 1998.
- S. Khuller, A. Moss y J. Naor. *The budgeted maximum coverage problem.* Information Processing Letters 70(1), 1999.
- M. Sviridenko. *A note on maximizing a submodular set function subject to a knapsack constraint.* Operations Research Letters 32(1), 2004.
- R. M. Karp. *Reducibility among combinatorial problems.* En *Complexity of Computer Computations*, 1972.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms* (montículos y algoritmos voraces).